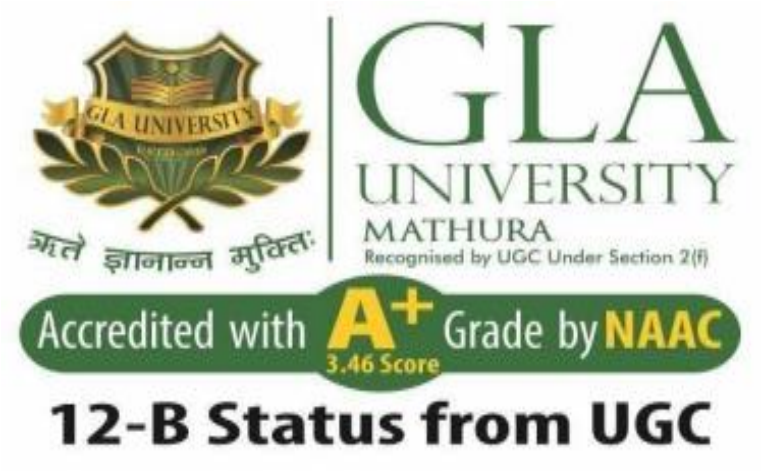
B.Tech CSE (III YEAR – VI SEM) (2026-27)

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING & APPLICATIONS

GLA UNIVERSITY



17km Stone, NH-2, Mathura-Delhi Road, P.O. Chaumuhan, Mathura – 281406 (Uttar Pradesh) India

---

Project Title: BEACON — A Behavior-Aware AI Companion for Academic Task Management and Opportunity Discovery

Team Leader: Aditi Sharma **UR:** 2415500026

Mentor Name: Sir Preshit Desai

Signature:

Project Synopsis:

BEACON — A Behavior-Aware AI Companion for Academic Task Management and Opportunity Discovery

Cover

- **Project title:** BEACON — A Behavior-Aware AI Companion for Academic Task Management and Opportunity Discovery
- **Team & ID:** T-1 (Solo Developer)
- **Institute / Course:** GLA University, Mini-Project
- **Version:** v1.0
- **Date:** 02 Sep 2026

1. Overview

Problem Statement

I have personally struggled with tracking tasks and opportunities scattered across WhatsApp, notices, and college portals, missing things like a free Microsoft certification opportunity and other hackathons simply because I wasn't tracking them well. Even when I do set deadlines for personal goals or club events, I often just forget about them — I see a task, mark it as seen, and lose track of it. Tools like Notion, sticky notes, and reminders eventually became background noise I started ignoring. Balancing my responsibilities alongside academics and exploring new opportunities has made this worse. Even when I do have all my tasks listed, I often get overwhelmed by having too many at once and don't know where to start, which ironically makes me delay starting on any of them, leaving assignments to the last minute.

Goal

Build BEACON, a desktop application (using Electron.js) that acts as an adaptive personal secretary for students. It automatically captures tasks and opportunities, prioritizes them based on real deadlines and user behavior patterns, and creates a complete, transparent roadmap. Beyond just listing tasks, the AI fetches relevant resources and suggests tools. The standout feature is its dynamic planning: if a student ignores a 5-day roadmap and only 2 days are left, BEACON detects the procrastination, scraps the old plan, and generates a new "last-minute" survival roadmap (e.g., suggesting one-shot videos or question banks). The main engagement feature is a floating pet avatar that stays on the desktop screen. Its tone is designed to feel like a friend rather than a nagging system, using casual language (like "Hey, that deadline's coming up, you've got this"). If the student keeps ignoring a task, the avatar's tone gradually shifts to strict to force action.

Non-Goals

- Native mobile apps — since this is a desktop application, v1 is completely out of scope for mobile.
- Autonomous task execution on behalf of the user.
- Institution-wide deployment.

Note: While the app can capture screens, continuous 24/7 background screen monitoring is out of scope to respect user privacy and system resources; the user will trigger the capture.

Value Proposition

Unlike static reminder apps, BEACON understands a student's actual workload and behavior. It doesn't just remind you of a deadline; it builds a transparent roadmap, fetches the resources you need, and explains the reasoning behind its suggestions. The desktop avatar adapts its tone and urgency based on how the user responds, instead of repeating the same reminder until it gets ignored.

---

2. Scope and Control

2.1 In-Scope (v1 Software Development Boundaries)

- **User Profile & Dynamic Skill Tagging:** Users can input their current skills initially. When a user completes a learning task, the AI will ask if they built a project. If the user submits a link or repo, the AI extracts the new skill and automatically updates the profile.
- **Task Management & Universal Capture:** Manual task creation and Google Calendar sync. Users can click the avatar to capture a screenshot from anywhere, or paste text, and the AI will extract the task and deadline.
- **Behavior-Aware Priority Engine:** AI that ranks tasks based on deadlines, complexity, and a dynamic procrastination score. It will give a short, transparent reason for the ranking.
- **AI Roadmap & Resource Assistant:** For complex tasks, the AI will break them into a step-by-step roadmap and automatically suggest useful tools or resource links.
- **Adaptive Planning (Personal Secretary Mode):** If a user wastes time and a deadline is closing in, the AI will detect the delay, scrap the original long roadmap, and generate a quick-survival plan (like suggesting one-shot videos or question banks) to ensure the student covers what is necessary.
- **Opportunity Discovery:** A basic scraper to find hackathons and workshops, matching them to the user's dynamically updated skills.
- **Desktop Floating Companion:** A web-based floating pet avatar that stays on the screen and tracks user inactivity.
- **Focus Mode (Activity Monitor):** User can trigger a "Focus Mode" or "Class Mode" where the app monitors active windows. Just like screen-sharing, the user can select which apps are allowed (whitelist). If the user opens a distracting app (like a game or Netflix) during

this time, the avatar detects it and gives a strict warning.
- **Adaptive Tone System:** The avatar's tone and notification frequency change based on the procrastination score or Focus Mode violations (friendly → nudging → strict).
- **Weekly Analytics Dashboard:** A simple screen showing how many tasks were completed on time versus delayed.

2.2 Out-of-Scope (What will NOT be built in v1)

- Native mobile apps. It will strictly be a desktop app for now.
- Continuous 24/7 video recording of the screen. The app will only check the names/titles of active windows during Focus Mode to respect privacy and save system resources.
- Institution-wide deployment or admin panels.

2.3 Assumptions

- Users will have a Google account for Calendar integration.
- Free-tier LLM inference (Groq) will be fast enough for the priority reasoning and image/text parsing.
- Opportunity platforms (Unstop, Devfolio) will allow basic, low-frequency scraping.

2.4 Constraints

- Strict 8-week academic timeline for a solo developer.
- Zero budget — relying entirely on free-tier APIs and hosting (Render for backend, Groq for AI).
- Rate limits on free LLM and third-party APIs.

2.5 Dependencies

- Groq API (Llama 3) for reasoning, roadmaps, tone generation, and parsing pasted text/screenshots.
- Google Calendar API (OAuth2) for syncing deadlines.
- External opportunity platforms (Unstop, Devfolio) for scraping event data.

2.6 Acceptance Criteria and Sign-Off

- GIVEN a user saves a new task WHEN the priority engine evaluates it THEN the system generates a priority score and a short reason in under 3 seconds.
- GIVEN a user snoozes a task multiple times WHEN the procrastination score rises THEN the pet avatar's tone changes to a stricter, more urgent message.
- GIVEN a user asks for a roadmap on a complex task WHEN the AI responds THEN it returns at least 3 actionable steps and suggests one relevant tool or link.

- GIVEN a user uploads any screenshot or pastes text WHEN the AI parses it THEN the task name, deadline, or resource is extracted for the user to confirm and save.
- GIVEN the opportunity scraper runs WHEN events are fetched THEN at least 3 relevant upcoming events are displayed based on the user's profile.
- GIVEN a user ignores a 5-day roadmap and only 2 days are left WHEN the AI detects the procrastination THEN it scraps the old plan and suggests a new quick-survival roadmap with different resources.
- GIVEN a user triggers Focus Mode WHEN the user opens a non-whitelisted app (like a game) THEN the avatar immediately shows a strict warning.

Sign-Off Table

Stakeholder	Role	Decision Area	Signature/Approval	Date
Mr. Preshit Desai	Mentor	Scope, final acceptance		
Aditi Sharma	Student Developer / Lead	Release readiness		

---

3. Stakeholders and RACI

Since this is an individual project, the solo student developer holds complete responsibility for execution, with Mr. Preshit Desai acting as the consulting and approving mentor.

- **R (Responsible):** The person who performs the activity.
- **A (Accountable):** The person with final decision-making power and veto rights.
- **C (Consulted):** Key advisors who provide input.
- **I (Informed):** People kept up-to-date on progress.

Activity	Responsible (R)	Accountable (A)	Consulted (C)	Informed (I)
Requirements	Aditi Sharma	Aditi Sharma	Preshit Desai Sir	—
Design	Aditi Sharma	Aditi Sharma	Preshit Desai Sir	—
Implementation	Aditi Sharma	Aditi Sharma	—	Preshit Desai Sir
Testing	Aditi Sharma	Aditi Sharma	Preshit Desai Sir	—
Release	Aditi Sharma	Aditi Sharma	Preshit Desai Sir	Dept

---

4. Team and Roles

Member	Role	Responsibilities	Key Skills	Availability	Contact
Aditi Sharma	Solo Developer (Product Lead, Full-Stack & AI)	Scope, architecture, backend, frontend, AI/prompt engineering, testing, docs	Django, React, Electron.js, LLM/Prompt Engineering	20+ hrs/wk	aditi.sharma_cs.aiml24@gla.ac.in

---

5. Week-wise Plan and Assignments

Week	Milestones & Deliverables	Primary Tasks (Solo Developer)	Deliverable Artifacts	Status
Weeks 1–2	Foundation & Core AI	Set up Django backend, PostgreSQL, and Electron + React skeleton. Implement JWT Auth, Task CRUD, and integrate Groq API. Build the Behavior-Aware	Backend repo, Electron app shell, Priority API, Capture flow.	Planned

Week	Milestones & Deliverables	Primary Tasks (Solo Developer)	Deliverable Artifacts	Status
		Priority Engine and Screenshot-to-Task capture.		
Weeks 3–4	Adaptive Planning & Companion	Build the AI Roadmap Assistant with dynamic logic (scrapping 5-day plan for a 2-day survival plan). Develop the floating pet avatar UI and link its tone to the procrastination score.	Roadmap API, Floating Avatar UI, Adaptive Tone system.	Planned
Weeks 5–6	Focus Mode & Opportunities	Implement Activity Monitor (Focus Mode) with app whitelist/blacklist. Build scraper for Unstop/Devfolio and implement Dynamic Skill Tagging from project links.	Focus Mode script, Scraper bot, Skill-extraction logic.	Planned
Weeks 7–8	Dashboard,	Build the	Final Desktop	Planned

Week	Milestones & Deliverables	Primary Tasks (Solo Developer)	Deliverable Artifacts	Status
	Hardening & Demo	Weekly Analytics Dashboard. Fix bugs, handle edge cases, prepare final presentation slides, and record the demo video.	App build, Demo deck, Documentation.	

---

6. Users and UX

6.1 Personas

- **The Overwhelmed Explorer:** A typical college student trying to balance regular academics while also looking for opportunities like hackathons, workshops, or internships. Because they are always exploring new things, they end up with too many tasks and need an AI to tell them what to prioritize right now.
- **The Procrastinating Student:** A student who gets paralyzed by seeing a long list of pending work and ends up doing nothing. They need the AI to break tasks into smaller steps and give them strict, escalating nudges to actually start working.

6.2 Top User Journeys

User logs in for the first time → App asks basic setup questions (current skills, academic goals, working habits) → User profile is created → User clicks the floating avatar to capture a screenshot or paste text → AI extracts task and deadline → AI prioritizes task and gives a step-by-step roadmap with resources → User triggers Focus Mode to study → If user opens a distracting app, avatar gives a strict warning → User completes task and submits project link → Profile skills update automatically → Dashboard shows new matched opportunities.

6.3 User Stories

- As a new user, I want the app to ask me about my skills and goals during the first login, so it can act as my personal guide from day one.
- As a student, I want to capture any screen deadline with one click, so I don't have to manually type and save tasks.
- As a student, I want the AI to scrap my 5-day plan and make a 2-day survival plan if I procrastinate, so I can still cover what's necessary before the exam.
- As a student, I want a Focus Mode that blocks distracting apps, so I can actually concentrate during class or study time.

6.4 Accessibility & Localization

- Simple, casual language used by the AI companion (no robotic or overly formal alerts).
- Desktop-oriented interface (Electron app) designed for focused studying.
- English language support for v1.

---

7. Market and Competitors

7.1 Competitor Table

Competitor	Product	Target Users	Key Features	Pricing	Strengths	Weaknesses	Our Differentiator
Google Calendar	Calendar & reminder service	General users	Event scheduling, reminders	Free	Ubiquitous, reliable sync	No prioritization, generic reminders	AI priority scoring with dynamic tone changes.
Notion	All-in-one workspace	Professionals, students	Manual organization, notes	Free	Highly customizable	Fully manual, high upkeep	Auto-capture from screenshots + AI roadmaps.
Todoist	Task list app	General users	Task lists, fixed reminders	Free	Simple, lightweight	Static reminders, easily ignored	Desktop avatar with adaptive tone.
Motion / Reclaim	AI auto-scheduling tools	Professionals	AI auto-scheduling, calendar sync	Paid ($19–$34/mo)	Dynamically plans the day	Very expensive, no student focus	Built specifically for students with opportunity discovery and free/low-cost.

Competitor	Product	Target Users	Key Features	Pricing	Strengths	Weaknesses	Our Differentiator
Forest / Cold Turkey	Focus timers & app blockers	Students, writers	Focus timers, app blockers	Free	Good for stopping distractions	No task management, just a blocker	Integrated Focus Mode linked to an AI avatar that actively nags you.

7.2 Positioning

BEACON is positioned as a proactive personal secretary, not just a to-do list. Its main motive is to entirely handle the mental load of planning for the student.
Measurable Delta:

- Task capture time: ≤10s via screenshot vs 1–2 min manual entry.
- AI Roadmap generation: ≤3s vs manually planning for hours.
- Engagement: Strict tone + Focus Mode vs generic push notifications that get ignored.

---

8. Objectives and Success Metrics

- **O1: Task Capture Accuracy** — AI successfully extracts task and deadline from pasted text/screenshots with ≥85% accuracy by Week 4. (KPI: extraction accuracy %)
- **O2: AI Latency** — Priority score and roadmap generation takes ≤3 seconds by Week 4.

(KPI: response time in seconds)

- **O3: Procrastination Reduction** — Measurable decrease in tasks completed <24h before deadline during the 4-week testing phase. (KPI: on-time completion rate)
- **O4: Focus Mode Compliance** — Avatar successfully detects and warns against 90% of blacklisted apps during active Focus Mode sessions. (KPI: detection rate)

---

9. Key Features

Feature	Description	Priority	Dependencies	Acceptance Criteria
Onboarding & User Profile	First-time login setup where AI asks questions to understand the user's skills, goals, and habits to build a base profile.	Must	Auth, DB	When a user logs in for the first time, they are prompted with setup questions, and their profile is saved successfully.
Universal Capture	Click avatar to screenshot any screen or paste text. AI extracts task/deadline.	Must	Groq API, Electron	When a user uploads a screenshot, the AI extracts task name and deadline for confirmation in

Feature	Description	Priority	Dependencies	Acceptance Criteria
				<5s.
Behavior-Aware Priority Engine	Ranks tasks on deadline, complexity, and dynamic procrastination score.	Must	Task DB, Groq API	When a user views tasks, they are ranked with a visible, transparent reason.
Adaptive Planning (Personal Secretary)	Dynamically changes roadmap from long-term to quick-survival (one-shot videos) if deadline is near.	Must	Groq API	If a user ignores a task and time is running out, the AI scraps the old plan and suggests new quick resources.
Focus Mode (Activity Monitor)	Desktop avatar monitors active windows against a whitelist and warns of distractions.	Should	Electron OS APIs	If a user opens a non-whitelisted app during Focus Mode, the avatar shows a strict warning.
Dynamic Skill Tagging	Extracts skills from project links/repos submitted by user to update profile.	Should	Groq API	When a user submits a repo link after a task, the AI extracts a skill and updates the

Feature	Description	Priority	Dependencies	Acceptance Criteria
				profile.
Opportunity Discovery	Scrapes Unstop/Devfolio and matches events to dynamically updated skills.	Should	Scraper, User Profile	When the scraper runs, it displays at least 3 relevant events based on the user's current skills.

---

10. Architecture

10.1 High-Level Architecture

BEACON follows a modular desktop architecture built on Electron.js, React, and Django.

- **Client Layer (Electron + React):** The desktop app is divided into two parts. The Main

Process (Electron) handles OS-level permissions, specifically the floating avatar window and the Focus Mode (tracking active windows). The Renderer Process (React) handles the UI, dashboard, and task views.

- **Capture & AI Layer:** When the user clicks the avatar to capture a screenshot or paste text, it is sent to the backend. The Groq LLM processes this text/image to extract the task name, deadline, and suggest resources.
- **Core AI Engine (Backend — Django):** This layer handles the logic. It calculates the dynamic procrastination score, generates the adaptive roadmaps (scrapping old plans if time runs out), and extracts skills from submitted project links.
- **Opportunity Layer:** A scheduled background script (cron job) scrapes Unstop and Devfolio periodically. It uses the LLM to match scraped events with the user's dynamically updated skill profile.
- **Data Layer:** PostgreSQL database to store Users, Tasks, Roadmaps, and Opportunities.

10.2 API Spec Snapshot

The React/Electron frontend will communicate with the Django backend via the following REST API endpoints:

Endpoint	Method	Auth	Purpose	Request Schema	Response Schema	Codes
/api/auth/register	POST	—	Create account	email, password	201 {id}	201, 400
/api/tasks/capture	POST	JWT	Upload screenshot / pasted text for AI extraction	image file / text string	{task_title, deadline}	200, 401
/api/tasks/prioritize	GET	JWT	Fetch AI-ranked task list with reasoning	—	{tasks[], priority_score, reason}	200

Endpoint	Method	Auth	Purpose	Request Schema	Response Schema	Codes
/api/tasks/roadmap	POST	JWT	Generate / update dynamic survival roadmap	task_id, plan_type	{steps[]}	200, 401
/api/focus/check	POST	JWT	Send active window name to check against whitelist	window_title	{allowed, warning_level}	200
/api/opportunities/match	GET	JWT	Fetch skill-matched hackathons / workshops	—	{events[]}	200

10.3 Configuration and Secrets

- Environment variables (DB credentials, API keys) managed via .env files in Django and Electron's main process.
- Google OAuth tokens for Calendar sync encrypted at rest.
- Groq LLM API keys stored securely in the backend, never exposed to the frontend client.

---

11. Data Design

11.1 Core Entities

- **User:** id (UUID, PK), email (String, Unique)
- **Task:** id (UUID, PK), title (String), priority_score (Decimal 0.0–10.0), deadline

(Timestamp)

- **Roadmap:** id (UUID, PK), plan_type (Enum: Full/Survival), steps (JSON list)
- **Opportunity:** id (UUID, PK), skill_tags (Array of strings)
- **Skill:** name (String, auto-extracted from project links)

11.2 Data Dictionary

Entity	Field	Type	Null?	Allowed Values	Source	Notes
User	id	UUID	No	—	System	Primary Key
User	email	String	No	RFC 5322	User	Unique
Task	id	UUID	No	—	System	Primary Key
Task	title	String	No	—	AI / User	Extracted from screenshot or text
Task	priority_score	Decimal	No	0.0 – 10.0	AI Engine	Calculated dynamically
Task	deadline	Timestamp	No	Date/Time	User / Calendar	Extracted from text/screenshot
Roadmap	id	UUID	No	—	System	Primary Key

Entity	Field	Type	Null?	Allowed Values	Source	Notes
Roadmap	plan_type	Enum	No	Full / Survival	AI Engine	Changes if user procrastinates
Roadmap	steps	JSON	No	List of strings	AI Engine	Step-by-step actions
Opportunity	id	UUID	No	—	System	Primary Key
Opportunity	skill_tags	Array	No	List of strings	Scraper	Used for matching with user profile
Skill	name	String	No	—	AI Engine	Auto-extracted from project links

11.3 Schemas and Migrations

- ER Diagram maintained in the appendix.
- Database migrations version-controlled using Django ORM.
- SQLite used for local desktop storage during development; PostgreSQL for backend hosting.

11.4 Privacy, Retention, and Backup

- PII (name, email) encrypted at rest; access restricted to the authenticated user.
- Focus Mode active window titles are checked locally in RAM and NOT stored in the

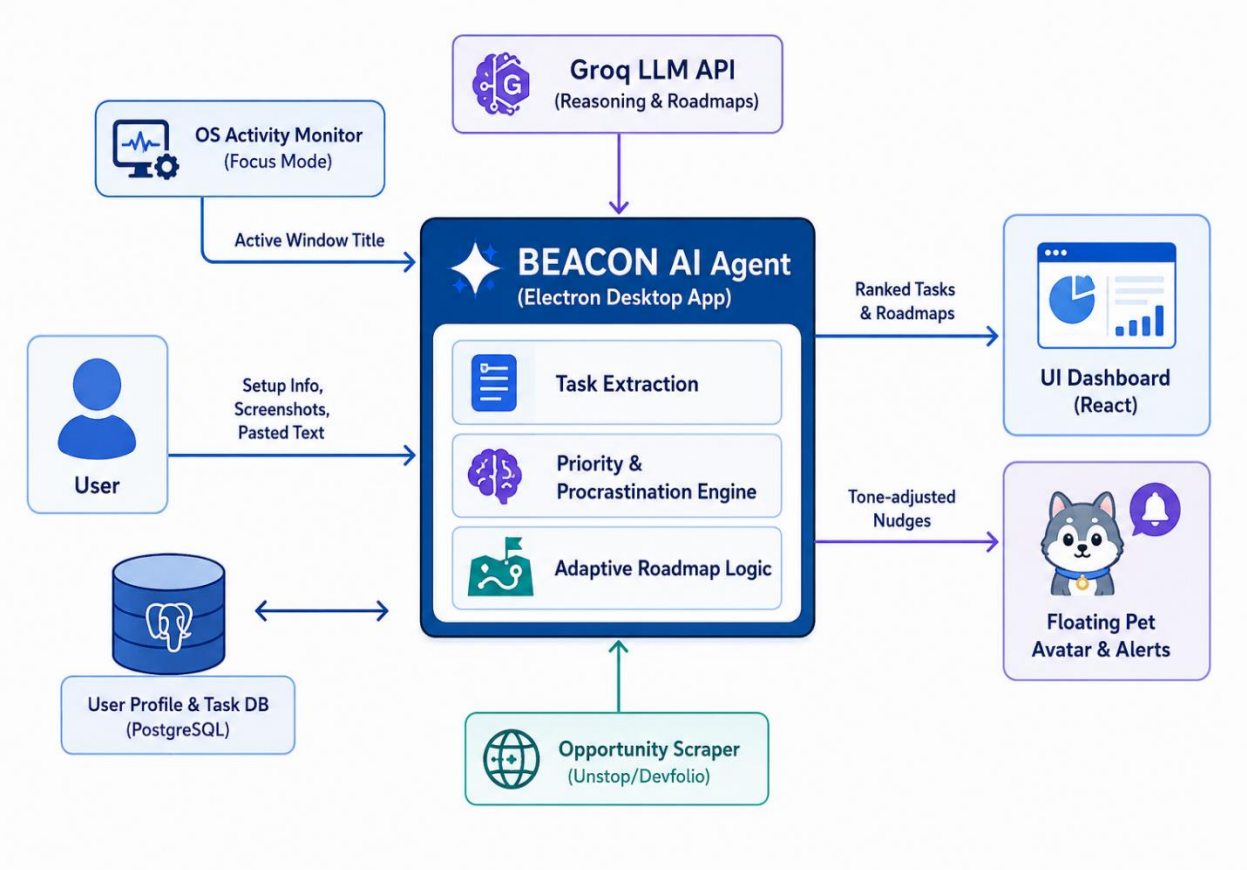
database to ensure strict privacy.
- Inactive accounts and their task data are purged after 12 months.
- Nightly automated backups during the active development and demo period.

---

12. Technical Workflow Diagrams

The following diagrams will be included in the project appendix to visually explain the system behavior, AI logic, and data flow:

- **System Architecture Diagram:** A layered view showing the Electron Desktop Client (React UI + OS-level Focus Mode), the Django Backend, and the Groq AI API connections.
- **Entity Relationship Diagram (ERD):** Shows the database structure and relationships between User, Task, Roadmap, Skill, and Opportunity entities.
- **Sequence Diagram (Task Capture):** A step-by-step flow showing how a user clicks the avatar, captures a screenshot, sends it to the backend, and how the Groq API extracts the task and deadline for user confirmation.
- **Sequence Diagram (Adaptive Roadmap):** Shows the logic flow of how the system detects a missed deadline, scraps the original 5-day plan, and requests a new 2-day survival roadmap from the LLM.
- **State Transition Diagram (Avatar Tone):** Shows how the pet avatar changes its state based on the procrastination score (Friendly → Nudging → Strict).



---

13. Quality: NFRs and Testing

13.1 Non-Functional Requirements

Metric	SLI	Target (SLO)	Measurement
Availability	Crash-free app sessions	Stable, crash-free operation through the 8-week development and final demo	Crash logs / run observation

Metric	SLI	Target (SLO)	Measurement
		period	
Latency	p95 AI response time	AI task extraction, priority scoring, and roadmap generation ≤ 3 seconds	API logs
Error rate	5xx %	< 2% backend API errors	Django logs
Privacy	Window titles stored/transmitted	Focus Mode checks active window titles locally in RAM only; never logged or transmitted	Code review / local audit

13.2 Test Plan

Area	Type	Tools	Owner	Coverage Target	Exit Criteria
Backend logic	Unit Testing	Django Tests / Pytest	Aditi Sharma	≥ 60%	No unresolved P1/P2 defects
Desktop UI	E2E / Component	React Testing Library	Aditi Sharma	Core task flows	All core flows pass
API	Integration	Postman	Aditi Sharma	All critical	All critical

Area	Type	Tools	Owner	Coverage Target	Exit Criteria
endpoints				endpoints	endpoints pass
AI & prompt logic	Manual Testing	Groq Playground / Logs	Aditi Sharma	Tone & extraction accuracy	Extraction accuracy ≥ 85%

13.3 Environments

- **Development Environment:** Local machine setup. Electron frontend and React UI running locally, Django backend running on localhost, and a local database for testing.
- **Testing Environment:** Backend temporarily deployed on Render (free tier) to test live API requests (Groq, Google Calendar, scrapers) and verify how the Electron app behaves when connecting to a real server.
- **Demo/Production Environment:** The final packaged Electron desktop application (built as .exe or .AppImage) connecting to the hosted Django backend for the final submission.

---

14. Security and Compliance

14.1 Threat Model (STRIDE)

Asset	Threat	STRIDE	Impact	Likelihood	Mitigation	Owner
Auth tokens (JWT)	Theft / Spoofing	Spoofing	High	Medium	HTTPS, short-lived JWTs, refresh token	Aditi Sharma

Asset	Threat	STRIDE	Impact	Likelihood	Mitigation	Owner
					rotation	
User data (Tasks/Skills)	Unauthorized access / SQLi	Tampering / Information Disclosure	High	Low	Parameterized queries, row-level access checks	Aditi Sharma
Focus Mode window data	Privacy leakage	Information Disclosure	High	Low	Process window titles locally in RAM; never log or transmit	Aditi Sharma
OAuth tokens (Google)	Token leakage	Information Disclosure	Medium	Low	Encrypted storage at rest; request minimal scopes	Aditi Sharma
Screenshot data (for capture)	Unintended retention	Information Disclosure	Medium	Low	Process via AI for text extraction; delete image post-extraction	Aditi Sharma

14.2 AuthN/AuthZ

- Email + password authentication.
- JWT-based authorization for securing backend API endpoints.
- Role-based access control ensuring users can only fetch and modify their own tasks and profiles.

14.3 Audit and Logging

- Log signups, logins, task CRUD operations, and AI API usage to monitor system behavior.
- 30-day retention policy for logs during the development and testing phase.

14.4 Compliance

- Academic mini-project; strictly follows institute data-handling policy.
- No third-party data sharing or selling.
- Screen monitoring is strictly opt-in and explicitly triggered by the user (Focus Mode or Screenshot Capture), ensuring the app never acts as background spyware.

---

15. Delivery and Operations

15.1 Release Plan

- Version v1.0 desktop application (Electron build) ready for final project submission.
- Incremental feature rollout across the 8-week timeline (Core AI → Companion Avatar → Opportunities).

15.2 CI/CD and Rollback

- CI pipeline: lint → test → build → deploy (using GitHub Actions for backend).
- Backend deployed on Render free tier.
- Rollback: revert to the previous stable Git commit and trigger a manual redeploy on Render.

15.3 Monitoring and Alerting

Metric	Threshold	Alert To	Runbook
p95 latency	> 3000 ms	Aditi Sharma	"AI Latency" runbook
Error rate	> 2%	Aditi Sharma	"Error Spike" runbook

15.4 Runbooks

- **AI Latency:** Check Groq API rate limits → reduce LLM prompt token size → clear cache → revert recent code change.
- **Error Spike:** Inspect Django logs → identify failing endpoint → fix bug → roll back to stable deployment.

15.5 Communication Plan

- Solo developer weekly self-review against the 8-week phase plan.
- Weekly status update and demo to mentor (Mr. Preshit Desai) every Friday.

---

16. Risks and Mitigations

16.1 Risk Heatmap

Risk	Probability	Impact	Score	Mitigation	Owner	Status
Schedule slip (8-week solo	Medium	High	12	Scope freeze, weekly	Aditi Sharma	Open

Risk	Probability	Impact	Score	Mitigation	Owner	Status
timeline)				demos, cut "Should" features first		
Groq API rate limits / downtime	Medium	Medium	9	Cache recent AI responses, use fallback LLM, size limits	Aditi Sharma	Open
Focus Mode OS restrictions	Low	High	8	Keep a fallback notification system if OS blocks window tracking	Aditi Sharma	Open
Screenshot OCR / text extraction inaccuracy	Medium	Medium	9	Confirm-before-save step so user can manually edit task	Aditi Sharma	Open

17. Research and Evaluation

17.1 Study of Existing Platforms

Platforms like Google Calendar, Notion, Todoist, and ChatGPT each address a slice of student productivity. However, they lack personalized, AI-driven advice based on student behavior. Users must manage tasks manually and interpret deadlines themselves, which can overwhelm students. Emotional bias mitigation (procrastination) and explainable AI are limited. This motivated BEACON, a proactive, behavior-aware desktop companion.

17.2 Evaluation Methodology

Self-testing across a 4-week personal trial period will compare task-completion timing before and after adopting BEACON's adaptive roadmaps and Focus Mode. The AI's dynamic planning (scrapping a 5-day plan for a 2-day survival plan) will be tested against missed deadlines.

17.3 User Feedback

College peers and club members will provide feedback on AI clarity, tone escalation, ease of interaction, and trustworthiness. Feedback will guide improvements in the avatar's nudging logic and usability.

17.4 KPI Tracking

Key metrics include AI response time (p95), screenshot extraction accuracy, reduction in last-minute submissions, and system uptime. Regular monitoring ensures reliability and continuous improvement.

---

18. Appendices

18.1 Glossary

- **AI:** Systems simulating human intelligence for reasoning and extraction.
- **KPI:** Metrics measuring system and project performance.
- **SLO:** Target service performance level (e.g., latency, uptime).
- **p95:** 95th percentile response time.
- **Electron:** Framework to build desktop apps using web technologies (React/Node.js).
- **Focus Mode:** Local OS-level monitor that checks active windows against a whitelist.

18.2 References

- Groq API Documentation: https://console.groq.com/docs
- Electron Documentation: https://www.electronjs.org/docs/latest
- Google Calendar API: https://developers.google.com/calendar
- React Documentation: https://react.dev
- Django REST Framework: https://www.django-rest-framework.org
- Unstop (Opportunity Source): https://unstop.com
